

8교시: 구름 EXP 배움일기

W4D3 Lesson 8

W4D2 에서 배포한 애플리케이션과 모니터링 스텔을 활용하여, 지표/대시보드/알람을 확인하고 학습 내용을 **기록**합니다.

Goorm EXP 관측(Observability) 저널 (Evidence Journal)

1. 모니터링 엔드포인트 정보

Prometheus URL

http://prometheus.monitoring.svc:9090 접근 OK

Grafana URL

http://grafana.monitoring.svc:3000 접근 OK

접속 계정

- Grafana ID : admin
- Prometheus (default)
- Grafana PW : ***** (변경 완료)
- Loki (logs) (설정 시)

2. 확인한 대시보드

- ✓ Kubernetes / Compute Resources / Namespace (v1)
- ✓ Kubernetes / Compute Resources / Pod (v1)
- ✓ Kubernetes / Compute Resources / Node (v1)
- ✓ kube-prometheus-stack / Grafana / Overview
- ✓ Kubernetes / API Server
- ✓ Alertmanager Overview

🔍 Tip: 좌측 메뉴 → Dashboards → Browse 에서 검색/선택

3. Targets 상태 (Prometheus → Status → Targets)

UP	28 (93.3%)	DOWN	2 (6.7%)	UNKNOWN	0
Job	Instance	Endpoint	State	Last Scrape	Error
kubelet	10.244.1.2:10250	/metrics/cadvisor	UP	8s ago	-
node-exporter	10.244.0.23:9100	/metrics	UP	7s ago	-
api	10.244.1.15:8080	/metrics	UP	6s ago	-
frontend	10.244.2.10:8080	/metrics	UP	5s ago	-
db	10.244.1.11:8080	/metrics	DOWN	15s ago	connection refused
custom-exporter	10.244.2.25:9100	/metrics	DOWN	20s ago	context deadline exceeded

🔍 Tip: DOWN 타겟은 우선 원인 파악 (서비스/엔드포인트/네트워크/애플리케이션)

4. PromQL 주요 쿼리 모음

목적	PromQL (예시)
Pod CPU 사용률	sum(rate(container_cpu_usage_seconds_total{namespace="namespace"}[5m])) by Pod
Pod Memory 사용량	sum(container_memory_working_set_bytes{namespace="namespace"}) by (pod)
Pod 재시작 횟수	sum(increase(kube_pod_container_status_restarts_total[1h])) by (pod)
Ready 상태 (가장)	sum(kube_pod_status_ready{condition="true", namespace="namespace"})
Pod NotReady (개)	sum(rate(http_requests_total{status="5.."}[5m])) / sum(rate(http_requests_total[5m]))
HTTP CPU 사용률	100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

💡 Tip: Grafana의 describe를 통해 현재 보인 시계열인 대시보드 재사용률이 높아집니다.

5. 문제 상황할 땐? 지표/그래프 근거

CrashLoopBackOff (재시작 급증)

Readiness Probe 실패 (Ready 감소)

CPU Pressure (CPU 사용률/Throttling)

Pod: api-7cd89f8c6f-abc12
합계: 28 회 (30m)
상태:

Service: api-svc
Ready: 1 / 3
엔드포인트: 1 개

Pod: worker-6d77b677-xxxy1
CPU 사용률: 1000m (1.0 core)
Limit: 1000m → Throttling 발생

6. Alerts(알람) 미리보기

PrometheusRule: pod-alerts

Alert	State	For	설명
PodHighCPU	PENDING	2m	CPU 사용률 > 80%
PodCrashLooping	FIRING	5m	재시작 3회 이상
PodNotReady	PENDING	1m	Ready 0 상태 지속
HighErrorRate	INACTIVE	-	5xx 비율 > 5%

💡 Tip: 표를 직접히 설정할때, 일사적 스크리프를 통한 알림을 줄이세요.

알람 상태 의미

- 🔴 **FIRING**
조건이 지속되어 알람 발생 중
- 🟡 **PENDING**
조건은 일시적만 for 시간 미충족
- 🟢 **INACTIVE**
조건이 거짓이거나 해제됨

7. 실습 증거(실행 & 명령어)

```
$ kubectl get pods -n app
$ kubectl describe pod api-7cd89f8c6f-abc12 -n app
$ kubectl logs api-7cd89f8c6f-abc12 -n app --previous
$ kubectl get svc, ep -n app
$ kubectl top pod -n app
$ curl -s http://api-svc.api.svc.cluster.healthz
$ kubectl get events --sort-by=-lastTimestamp
$ kubectl -n monitoring get servicemonitors/podmonitors
$ kubectl -n monitoring port-forward svc/prometheus-operated 9090:9090
```

💡 Tip: --previous, describe, events 등을 통해 원인 파악이 훨씬 편리합니다.

8. 타임라인 요약

- 14:10 배포/일일 업데이트 시작
- 14:16 api Pod 재시작 증가시작
- 14:18 Readiness Probe 실패 증가
- 14:19 Service Endpoint 잔존 (3 → 1)
- 14:21 5xx 에러 비율 상승
- 14:22 알람 FIRING (PodCrashLooping)
- 14:25 물럭/수정 배포 수정

9. 학습 회고 (Reflection)

- ① 어떤 실습에서 파악한 주요 원인은 무엇이었나요?
설정 오류 (DB_HOST 오타)로 CrashLoopBackOff 발생
- ② 어떤 지표/로그가 가장 결정적 단서였나요?
increase(restarts_total) < 이전 전체타이틀 로그
- ③ 알람 설계에서 개선할 점은 무엇이었나요?
for 시간 조정, 일제적 세분화 노이즈 감소
- ④ 다음에 추가로 보고 싶은 대시보드들은?
애플리케이션 레이턴스, 요청량, DB 연결 수

10. 다음 단계 (W4D4 준비 질문)

- RBAC 최소 권한 원칙을 적용하지 않았다면 어떤 위험이 있을까요?
- NetworkPolicy / Kyverno 정책을 적용하면 이 문제해 예방할 수 있을까요?
- 자동화된 알람 약제(Silence) 정책은 어떤 기준으로 설정하면 좋을까요?

정리(결정)

이번 문제는 코드상 설정 수정 제때로 → 정상 복구 증가 수립 완료! 다음 단계로 이동합니다.

핵심 요약

- ✓ 관측은 빠른 원인파악의 핵심
- ✓ 지표/로그/이벤트를 함께 보면 정확도가 높아짐
- ✓ 알람은 적절한 임계값과 for 시간으로 설계
- ✓ 증거를 남기고 회고하면 재발을 줄일 수 있다

수업 목표

- Prometheus/Grafana에서 확인한 지표를 evidence로 정리한다.
- target down, restart 증가, readiness 실패를 구분해 기록한다.
- W4D4 RBAC/Kyverno로 이어질 운영 질문을 남긴다.

오늘 배운 내용 요약

주제	핵심
Observability	logs/events/metrics/traces를 조합해 장애 질문을 좁힘
Prometheus	target scrape와 PromQL
Grafana	dashboard로 시간 흐름 확인
ServiceMonitor	Service 기반 target discovery
Alert	사람이 개입할 조건 정의
Runbook	증상별 확인 순서와 전달 정보

배움일기 표

항목	기록
Prometheus 접속 URL	
Grafana 접속 URL	
확인한 dashboard	
UP target 예시	
DOWN 또는 헛갈린 target	
사용한 PromQL 1개	
CrashLoop metric	
readiness 실패 evidence	
CPU 압박 evidence	
alert preview 상태	
개발팀에 전달할 정보	

오늘의 evidence 명령

```
helm list -n monitoring
kubectl -n monitoring get pod,svc
kubectl get servicemonitor,podmonitor,prometheusrule -A
kubectl -n week4-observe get pod
kubectl -n week4-observe describe pod -l app=crashloop-demo
kubectl -n week4-observe describe pod -l app=readiness-bad-demo
```

PromQL:

```
up
increase(kube_pod_container_status_restarts_total[5m])
sum by (namespace, pod) (rate(container_cpu_usage_seconds_total{container!="",
image!=""}[5m]))
kube_pod_status_ready{condition="true"}
```

작성 예시

항목	기록 예시
확인한 dashboard	Kubernetes / Compute Resources / Pod
UP target 예시	kube-state-metrics
PromQL	increase(kube_pod_container_status_restarts_total[5m])
CrashLoop evidence	restart 증가 + logs --previous
readiness evidence	새 Pod READY 0/1 + readiness 404 event + 기존 Pod 유지
CPU evidence	cpu-pressure-demo CPU rate 증가
alert 상태	Week40observePodRestarting firing

실제 검증 기록 예시

항목	기록
cluster	kind, single control-plane node Ready
monitoring Pod	Prometheus 2/2, Grafana 3/3, Alertmanager 2/2 Running
CrashLoop	crashloop-demo restart 증가, alert firing
Readiness	nginx /not-ready 404로 새 Pod Ready 0/1
Target	local kind에서 일부 control plane target은 up=0 가능
Alert	ALERTS{alertname="Week40observePodRestarting"} 값 1

좋은 기록과 아쉬운 기록

아쉬운 기록	좋은 기록
Grafana 봤다	Pod dashboard에서 cpu-pressure-demo CPU 증가 확인
alert 떴다	Week40observePodRestarting firing, restart 증가 PromQL 확인
target이 이상했다	/targets에서 kube-state-metrics DOWN, error message 기록
Pod가 죽었다	CrashLoopBackOff + logs --previous 출력 + restart metric 증가
readiness 문제	READY 0/1 + event 404 + endpoint 없음

최종 runbook 한 문단

장애가 발생하면 먼저 사용자 증상과 시각을 기록한다. Prometheus target이 UP인지 확인하고, Grafana에서 namespace와 Pod 단위 resource/restart/ready 상태를 본다. 그다음

kubectl로 Service/Endpoint/Pod event/logs를 확인한다. metric은 시간과 범위를 보여주고, event와 log는 원인을 좁힌다. alert는 사람이 개입해야 할 조건만 남기고 noise가 되면 threshold와 for 시간을 조정한다.

오늘의 final note

W4D3에서는 kube-prometheus-stack을 Helm으로 설치하고 Prometheus target, Grafana dashboard, alert preview를 확인했다. target이 DOWN이면 dashboard보다 먼저 ServiceMonitor, Service, Endpoint, Pod readiness를 확인해야 한다. restart 증가, readiness 실패, CPU 압박은 각각 다른 metric과 kubectl evidence로 확인해야 한다. 좋은 alert는 단순 threshold가 아니라 사람이 해야 할 행동과 runbook으로 이어져야 한다.

W4D4로 이어지는 질문

질문	W4D4 연결
누가 monitoring namespace를 수정할 수 있는가	RBAC
누구에게 PrometheusRule 생성 권한을 줄 것인가	Role/RoleBinding
위험한 manifest를 배포 전에 막을 수 있는가	Kyverno
label/resource/probe 없는 Pod를 차단할 수 있는가	admission policy
policy 위반도 dashboard/alert로 볼 수 있는가	observability + security

개인 정리 질문

아래 질문에 답하면 오늘 수업의 핵심을 거의 정리한 것이다.

질문	내 답
up=0과 query 결과 없음은 어떻게 다른가	
ServiceMonitor에서 port name이 틀리면 어떤 일이 생기는가	
restart 증가를 볼 PromQL은 무엇인가	
readiness 실패는 restart metric에 반드시 보이는가	
rollout 중 이전 Pod가 남아 있는 이유는 무엇인가	
dashboard screenshot에 time range가 필요한 이유는 무엇인가	
alert의 for는 왜 필요한가	
alert noise를 줄이는 방법은 무엇인가	

수업 후 남길 파일/메모

```
# W4D3 observability memo

## Dashboards
- Prometheus URL:
- Grafana URL:
- 확인한 dashboard:

## Queries
- target:
- restart:
- CPU:
- readiness:

## Incident practice
- 증상:
- metric:
- kubectl evidence:
- 판단:

## Next
- W4D4에서 권한/정책으로 막아보고 싶은 위험:
```

cleanup 결정

선택	기준
monitoring stack 유지	W4D4에서도 dashboard/alert를 보고 싶음
monitoring stack 삭제	local resource를 아껴야 함
scenario namespace 삭제	CrashLoop/CPU loop가 계속 돌면 방해됨

삭제:

```
kubectl delete namespace week4-observe
helm uninstall kube-prometheus-stack -n monitoring
kubectl delete namespace monitoring
```

유지한다면 `helm list -A`와 `kubectl get ns` 결과를 기록한다.

Evidence Note

W4D3S8 Journal

- 오늘 가장 도움이 된 **dashboard**:
- 가장 헛갈린 **PromQL**:
- **target down**을 보면 먼저 확인할 것:
- **alert noise**를 줄이는 방법:
- W4D4에서 확인하고 싶은 권한/정책 질문:
- **monitoring stack** 유지/삭제 결정:

한 줄 요약

W4D3의 산출물은 **dashboard** 캡처가 아니라 장애를 좁히는 **metric** 질문과 **runbook**이다.